

lsh部分修改意见

【代码块 1.1：修复相关性分析】

- **用途：** 修正相关性计算的逻辑错误。
- **替换位置：** 替换掉原来计算相关性并输出到Markdown的部分。

code Python

downloadcontent_copy

expand_less

代码块

```
1      # CODE BLOCK 1.1: FIX CORRELATION ANALYSIS
2      report.append("### 1.3 Parameter Impact Analysis")
3      report.append("**Impact of Number of Tables:**")
4      # Fix: Use correlation on original data, not grouped means
5      report.append(f"- Build time correlation with table count:
6      {param_df['num_tables'].corr(param_df['build_time']):.3f}")
7      report.append(f"- Query time correlation with table count:
8      {param_df['num_tables'].corr(param_df['avg_query_time_ms']):.3f}")
9      report.append(f"- Recall correlation with table count:
10     {param_df['num_tables'].corr(param_df['recall_at_10']):.3f}\n")
11
12     report.append("**Impact of Hash Size:**")
13     report.append(f"- Build time correlation with hash size:
14     {param_df['hash_size'].corr(param_df['build_time']):.3f}")
15     report.append(f"- Query time correlation with hash size:
16     {param_df['hash_size'].corr(param_df['avg_query_time_ms']):.3f}")
17     report.append(f"- Recall correlation with hash size:
18     {param_df['hash_size'].corr(param_df['recall_at_10']):.3f}\n")
```

【代码块 1.2：修正时间复杂度估计】

- **用途：** 使用更严谨的方法估算时间复杂度。
- **替换位置：** 替换掉原来计算时间复杂度并输出到Markdown的部分。

code Python

downloadcontent_copy

expand_less

代码块

```
1      # CODE BLOCK 1.2: FIX TIME COMPLEXITY ESTIMATION
2      report.append("### 2.2 Scalability Conclusions")
3
4      # Better complexity estimation by using only larger data sizes
5      large_scale_df = scale_df[scale_df['data_size'] >= 10000]
6
7      if len(large_scale_df) >= 3:
8          log_sizes = np.log(large_scale_df['data_size'])
9          log_bf_times = np.log(large_scale_df['bf_avg_query_time_ms'])
10         log_lsh_times = np.log(large_scale_df['lsh_avg_query_time_ms'])
11
12         bf_time_growth = np.polyfit(log_sizes, log_bf_times, 1)[0]
13         lsh_time_growth = np.polyfit(log_sizes, log_lsh_times, 1)[0]
14
15         if bf_time_growth < 0.8 or bf_time_growth > 1.2:
16             report.append(f"- Brute force time complexity: O(n) [theoretical,
17 measured: O(n^{bf_time_growth:.2f})]")
18         else:
19             report.append(f"- Brute force time complexity estimate:
20 O(n^{bf_time_growth:.2f})")
21
22         report.append(f"- LSH time complexity estimate:
23 O(n^{lsh_time_growth:.2f})")
24     else:
25         report.append("- Insufficient data points for complexity analysis")
26
27     speedup_at_max = scale_df.loc[scale_df['data_size'].idxmax(),
28 'speedup_factor']
29     report.append(f"- At maximum tested size: {speedup_at_max:.2f}x speedup\n")
```

Part 2: 增加深度分析 (In-depth Analysis)

目标：“解剖” LSH，从三个维度诊断性能问题。

负责人：负责 `parameter_analysis_en.py` 的同学。

操作：将以下三个函数完整地复制到 `parameter_analysis_en.py` 脚本中，并确保在 `main()` 函数的末尾调用它们。

【代码块 2.1：LSH桶分布统计函数】

- **用途：** 分析数据倾斜问题。

code Python

downloadcontent_copy

expand_less

代码块

```
1      # CODE BLOCK 2.1: LSH BUCKET DISTRIBUTION ANALYSIS FUNCTIONdef
analyze_lsh_bucket_distribution(param_df, vectors):
2      """Analyze LSH bucket distribution characteristics"""
3      print("\n=== LSH Bucket Distribution Analysis ===")
4      best_config = param_df.loc[param_df['recall_at_10'].idxmax()]
5      lsh = LSHIndex(hash_family='random_projection',
6                      num_tables=int(best_config['num_tables']),
7                      hash_size=int(best_config['hash_size']))
8      lsh.build_index(vectors)
9
10     bucket_distribution = [len(bucket_size) for table in lsh.hash_tables for
bucket_size in table.values()]
11
12     stats = {
13         'min_bucket_size': np.min(bucket_distribution), 'max_bucket_size':
np.max(bucket_distribution),
14         'median_bucket_size': np.median(bucket_distribution),
15         'std_bucket_size': np.std(bucket_distribution),
16         'empty_buckets_ratio': sum(1 for size in bucket_distribution if size
== 0) / len(bucket_distribution),
17         'oversized_buckets_ratio': sum(1 for size in bucket_distribution if
size > 1000) / len(bucket_distribution)
18     }
19     plt.figure(figsize=(12, 5))
20     plt.subplot(1, 2, 1)
21     plt.hist(bucket_distribution, bins=50, edgecolor='black');
plt.xlabel('Bucket Size'); plt.ylabel('Frequency')
22     plt.title('LSH Bucket Size Distribution'); plt.yscale('log')
23
24     plt.subplot(1, 2, 2)
25     percentiles = [50, 75, 90, 95, 99]; values =
[ np.percentile(bucket_distribution, p) for p in percentiles]
26     plt.bar(range(len(percentiles)), values);
plt.xticks(range(len(percentiles)), [f'P{p}' for p in percentiles])
27     plt.ylabel('Bucket Size'); plt.title('Bucket Size Percentiles')
28     plt.tight_layout();
plt.savefig('results/analysis/bucket_distribution.png', dpi=300)
29     return stats
```

【代码块 2.2：候选集大小与性能分析函数】

- **用途：**分析“海选”范围对速度和精度的影响。

code Python

downloadcontent_copy

expand_less

代码块

```
1      # CODE BLOCK 2.2: CANDIDATE SET SIZE VS PERFORMANCE ANALYSIS FUNCTIONdef
analyze_candidate_set_performance(param_df, vectors, labels):
2      """Analyze relationship between candidate set size and performance"""
3      print("\n=== Candidate Set Size vs Performance Analysis ===")
4      results = []
5      test_queries = vectors[np.random.choice(len(vectors), 100, replace=False)]

6
7      for _, config in param_df.iterrows():
8          lsh = LSHIndex(hash_family='random_projection',
num_tables=int(config['num_tables']), hash_size=int(config['hash_size']))
9          lsh.build_index(vectors)
10         candidate_sizes, query_times = [], []
11
12         for query in test_queries:
13             start = time.time()
14             candidates = lsh._get_candidates(query) # Assumes _get_candidates
method exists in LSHIndex
15             query_time = (time.time() - start) * 1000
16             candidate_sizes.append(len(candidates));
query_times.append(query_time)
17
18             results.append({
19                 'num_tables': config['num_tables'], 'hash_size':
config['hash_size'],
20                 'avg_candidate_size': np.mean(candidate_sizes),
'std_candidate_size': np.std(candidate_sizes),
21                 'avg_query_time': np.mean(query_times), 'recall':
config['recall_at_10']
22             })
23
24     df = pd.DataFrame(results)
25     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```

26     axes[0].scatter(df['avg_candidate_size'], df['avg_query_time'],
c=df['num_tables'], cmap='viridis', s=100, alpha=0.6)
27     axes[0].set_xlabel('Average Candidate Set Size');
axes[0].set_ylabel('Average Query Time (ms)'); axes[0].set_title('Candidate
Set Size vs Query Time')
28     axes[1].scatter(df['avg_candidate_size'], df['recall'], c=df['hash_size'],
cmap='plasma', s=100, alpha=0.6)
29     axes[1].set_xlabel('Average Candidate Set Size');
axes[1].set_ylabel('Recall@10 (%)'); axes[1].set_title('Candidate Set Size vs
Recall')
30     axes[2].scatter(df['recall'], df['avg_query_time'],
c=df['avg_candidate_size'], cmap='coolwarm', s=100, alpha=0.6)
31     axes[2].set_xlabel('Recall@10 (%)'); axes[2].set_ylabel('Average Query
Time (ms)'); axes[2].set_title('Recall-Speed Tradeoff')
32     plt.tight_layout();
plt.savefig('results/analysis/candidate_set_analysis.png', dpi=300)
33     return df

```

【代码块 2.3：数据特征影响分析函数】

- **用途：** 分析数据本身的特性，诊断“先天”问题。

code Python

downloadcontent_copy

expand_less

代码块

```

1     # CODE BLOCK 2.3: DATA CHARACTERISTICS IMPACT ANALYSIS FUNCTIONdef
analyze_data_characteristics_impact(vectors):
2     """Analyze how data characteristics affect LSH performance"""
3     print("\n=== Data Characteristics Impact Analysis ===")
4     norms = np.linalg.norm(vectors, axis=1); pairwise_distances = []
5     sample_size = min(1000, len(vectors)); sample_indices =
np.random.choice(len(vectors), sample_size, replace=False)
6
7     for i in range(0, len(sample_indices), 10):
8         distances = np.linalg.norm(vectors - vectors[sample_indices[i]],
axis=1)
9         pairwise_distances.extend(distances[:100])
10
11     characteristics = {'vector_dimension': vectors.shape[1], 'num_vectors':
len(vectors), 'avg_norm': np.mean(norms),
12                        'std_norm': np.std(norms), 'avg_pairwise_distance':
np.mean(pairwise_distances),

```

```

13         'std_pairwise_distance': np.std(pairwise_distances)}
14
15     fig, axes = plt.subplots(1, 3, figsize=(15, 4))
16     axes[0].hist(norms, bins=50, edgecolor='black', alpha=0.7);
    axes[0].set_xlabel('Vector Norm'); axes[0].set_ylabel('Frequency');
    axes[0].set_title('Vector Norm Distribution')
17     axes[1].hist(pairwise_distances, bins=50, edgecolor='black', alpha=0.7);
    axes[1].set_xlabel('Pairwise Distance'); axes[1].set_ylabel('Frequency');
    axes[1].set_title('Pairwise Distance Distribution')
18     axes[2].text(0.1, 0.9, f"Dimension:
{characteristics['vector_dimension']}", transform=axes[2].transAxes)
19     axes[2].text(0.1, 0.8, f"Count: {characteristics['num_vectors']:,}",
    transform=axes[2].transAxes)
20     axes[2].text(0.1, 0.7, f"Avg Norm: {characteristics['avg_norm']:.2f}",
    transform=axes[2].transAxes)
21     axes[2].text(0.1, 0.6, f"Avg Dist:
{characteristics['avg_pairwise_distance']:.2f}", transform=axes[2].transAxes)
22     axes[2].axis('off'); axes[2].set_title('Data Characteristics')
23     plt.tight_layout();
    plt.savefig('results/analysis/data_characteristics.png', dpi=300)
24     return characteristics

```

Part 3: 实施改进方案 (Implement Improvements)

目标： 将LSH升级，实现至少一种更高级的策略。

负责人： 负责 `lsh_search.py` 或新建文件的同学。

操作： 以下是三种方案，**至少选择一种**来实现。

【代码块 3.1： 方案1 - 自适应参数调整】

- **用途：** 自动寻找“性价比”最高的LSH参数。
- **实现位置：** 可以放在 `parameter_analysis_en.py` 中作为一个独立的实验，或者封装成一个 `AdaptiveLSH` 类。

code Python

downloadcontent_copy

expand_less

代码块

```

1     # CODE BLOCK 3.1: ADAPTIVE PARAMETER TUNINGclass AdaptiveLSH:
2     """LSH with adaptive parameter tuning based on data characteristics"""

```

```

3     def __init__(self, target_recall=0.95, max_query_time_ms=50):
4         self.target_recall, self.max_query_time_ms = target_recall,
max_query_time_ms; self.optimal_params = None
5
6     def tune_parameters(self, vectors, sample_size=500):
7         print("=== Adaptive Parameter Tuning ===")
8         sample_indices = np.random.choice(len(vectors), min(sample_size,
len(vectors)), replace=False)
9         sample_vectors, query_vectors = vectors[sample_indices],
sample_vectors[:100]
10        param_grid = [{'num_tables': 5, 'hash_size': 8}, {'num_tables': 10,
'hash_size': 10},
11                        {'num_tables': 15, 'hash_size': 12}, {'num_tables': 20,
'hash_size': 14}]
12        best_config, best_score = None, float('-inf')
13
14        for params in param_grid:
15            lsh = LSHIndex(hash_family='random_projection', **params);
lsh.build_index(vectors)
16            start = time.time()
17            distances, indices = lsh.search(query_vectors, 10, 'cosine');
query_time = ((time.time() - start) / len(query_vectors)) * 1000
18            recall = 100.0 # Placeholder for actual recall calculation
if
query_time <= self.max_query_time_ms and recall >= self.target_recall:
19                score = recall / query_time
20                if score > best_score: best_score, best_config = score, params
21
22        self.optimal_params = best_config if best_config else param_grid[1]
print(f"Optimal parameters found: {self.optimal_params}"); return
self.optimal_params

```

- **注意:** 上述代码中的 `recall = 100.0` 是一个占位符。你需要在这里实现一个真实的、小范围的召回率计算（例如，与暴力搜索对比）来让这个调优过程真正有效。

【代码块 3.2：方案2 - 混合LSH + FAISS策略 (推荐!)】

- **用途:** LSH海选 + Faiss精排，工业界常用策略。
- **实现位置:** 建议创建一个新文件 `src/hybrid_search.py`。

code Python

downloadcontent_copy

expand_less

```

1      # CODE BLOCK 3.2: HYBRID LSH + FAISS STRATEGYimport faiss
2
3      class HybridIndex:
4          """Hybrid indexing using LSH for filtering and FAISS for refinement"""
5          def __init__(self, num_tables=10, hash_size=10):
6              self.lsh = LSHIndex(hash_family='random_projection',
7                                  num_tables=num_tables, hash_size=hash_size)
8              self.faiss_index = None
9
10             def build_index(self, vectors):
11                 print("Building hybrid index..."); self.lsh.build_index(vectors)
12                 dim = vectors.shape[1]; self.faiss_index = faiss.IndexFlatL2(dim)
13                 self.faiss_index.add(vectors.astype('float32')); print("FAISS index
14                 built successfully")
15
16             def search(self, query_vectors, k=10):
17                 all_distances, all_indices = [], []
18                 for query in query_vectors:
19                     candidates = self.lsh._get_candidates(query)
20                     if not candidates: candidates = list(range(min(k*10,
21                             self.lsh.vectors.shape[0])))
22
23                     # [PROFESSIONAL IMPLEMENTATION using IDSelector]
24                     candidate_selector =
25                     faiss.IDSelectorArray(np.array(list(candidates), dtype='int64'))
26                     search_params = faiss.SearchParameters(sel=candidate_selector)
27
28                     D, I = self.faiss_index.search(query.reshape(1,
29                             -1).astype('float32'), k, params=search_params)
30
31                     all_distances.append(D[0]); all_indices.append(I[0])
32                 return np.array(all_distances), np.array(all_indices)

```

- **注意:** 我在这里提供了比老师示例中更专业、更高效的基于 **IDSelector** 的FAISS精排实现，这是一个可以让你项目加分的细节。

【代码块 3.3：方案3 - 桶大小平衡优化】

- **用途:** 解决数据倾斜问题。
- **实现位置:** 需要修改现有的 **LSHIndex** 类的 **build_index** 方法。

code Python

downloadcontent_copy

expand_less

代码块

```
1      # CODE BLOCK 3.3: BUCKET SIZE BALANCING# This code should replace the
      existing build_index method in your LSHIndex class
from collections import
defaultdict
2
3  def build_index_with_balancing(self, vectors, max_bucket_size=500):
4      print(f"Building LSH index with bucket balancing (max size:
      {max_bucket_size})...")
5      self.vectors = vectors.astype(np.float32); dim = vectors.shape[1]
6      if self.hash_family == 'random_projection': self.hash_functions =
      self._generate_random_projection_functions(dim)
7      self.hash_tables = [defaultdict(list) for _ in range(self.num_tables)];
      overflow_vectors = []
8
9      for idx, vector in enumerate(tqdm(vectors, desc="Hashing with
      balancing")):
10         inserted = False
11         for table_idx in range(self.num_tables):
12             hash_key = self._hash_vector_random_projection(vector, table_idx)
13             if len(self.hash_tables[table_idx][hash_key]) < max_bucket_size:
14                 self.hash_tables[table_idx][hash_key].append(idx); inserted =
      True; break
15         if not inserted: overflow_vectors.append(idx)
16
17     if overflow_vectors:
18         print(f"Creating secondary storage for {len(overflow_vectors)}
      overflow vectors")
19         self.overflow_vectors = set(overflow_vectors) # Store indices of
      overflow vectors
20     self.is_built = True
```

- **注意:** 这段代码在 `search` 时还需要相应的逻辑来处理 `overflow_vectors`（例如，在LSH搜索后，总是将溢出集也加入候选集进行暴力搜索）。

这份完整的代码清单应该能覆盖老师提出的所有修改意见了。请分发给你的团队成员，并根据你们的时间和精力，**至少完成Part 1和Part 2**，并**至少选择一种Part 3的方案**来实现。